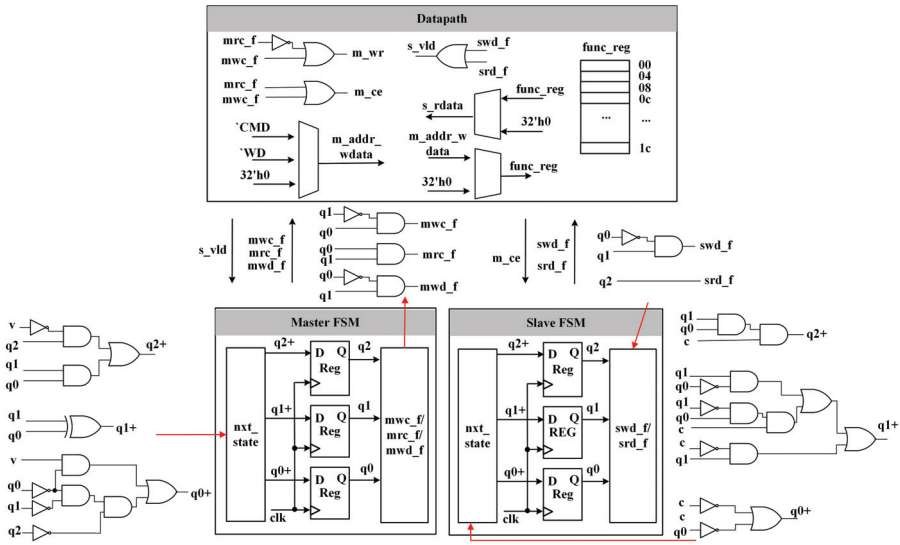

FIGURE 8.28
MSBUS master-slave circuit.

the next state transition. The sequential circuit comprises state registers that maintain the machine's current state. Likewise, the slave FSM generates flag signals ("swd_f" and "srd_f") through the combinational output circuit within the state machine and employs a combinational circuit for determining the next state transition. The sequential circuit is composed of state registers responsible for storing the current state.

Within the datapath, the master-slave communication adheres to the MSBUS protocol. The master datapath is responsible for transmitting essential command and data signals, which encompass "m_ce", "m_wr", and "m_addr_wdata" to the slave. Once these commands are received, the slave datapath issues corresponding signals, including "s_vld" and "s_rdata". It's worth noting that the slave datapath oversees the management of the register file, facilitating both data writing and reading via the MSBUS.

T&F-Not for Distri

Exercises

Problem 8.1. A 16-byte memory block is depicted in Figure 8.29, showing both addresses and data bytes. Assuming the use of a big-endian mode data bus to read the memory block, please consider the following questions:

- What data can be read out on the data bus corresponding to the configured memory addresses $0x00$, $0x01$, $0x02$, and $0x03$ when the bus transfer size is one byte?

- When the bus transfer size is half-word, what data can be read out on the data bus corresponding to the configured memory addresses $0x04$, $0x05$, $0x06$, and $0x07$?
- If the bus transfer size is a word, what data can be read out on the data bus corresponding to the configured memory addresses $0x08$, $0x09$, $0x0a$, and $0x0b$?

Assuming the use of a little-endian mode data bus to access the memory block, please consider the three questions again.

Memory address:	00	01	02	03	04	05	06	07	08	09	0a	0b	0c	0d	0e	0f
Memory data:	a0	a1	a2	a3	b0	b1	b2	b3	c0	c1	c2	c3	d0	d1	d2	d3

FIGURE 8.29
Memory block access through data bus.

T&F–Not for Distribution

FP Design Projects

In this textbook, all single-precision FP operators, such as FP adders and multipliers, are readily accessible as pre-designed intellectual properties (IPs). These RTL designs are generated using the Chisel hardware description language. Two versions of the design are provided:

- **Basic design version:** These designs offer single-clock cycle latency between inputs and outputs.
- **Pipelined design version:** The pipelined designs can achieve a higher operational clock frequency compared to the basic designs, although they require more clock cycles for each data computation.

Hint: 1) It's crucial to recognize that the design IPs are activated by the rising clock edge and rising reset edge. Consequently, it's advisable to integrate the IPs within the system constructed with a rising clock edge and rising reset edge for optimal performance and compatibility. 2) Moreover, when dealing with FP operations in digital circuits, it is essential to conform to the IEEE 754 format for the representation of FP numbers. To ensure precise representation and conversion, please consult Table 8.2 for the numbers referenced.

TABLE 8.2
IEEE 754 format for FP numbers.

FP	**1.0**	**2.0**	**3.0**	**4.0**	**5.0**	**6.0**
HW	3f800000	40000000	40400000	40800000	40a00000	40c00000
FP	**7.0**	**8.0**	**9.0**	**10.0**	**11.0**	**12.0**
HW	40e00000	41000000	41100000	41200000	41300000	41400000
FP	**-1.0**	**-2.0**	**-3.0**	**-4.0**	**-5.0**	**-6.0**
HW	bf800000	c0000000	c0400000	c0800000	c0a00000	c0c00000
FP	**-7.0**	**-8.0**	**-9.0**	**-10.0**	**-11.0**	**-12.0**
HW	c0e00000	c1000000	c1100000	c1200000	c1300000	c1400000

Not for Distribution

PBL 18: FP Add-Mul-Sub Design with FSMD

1) Design an FSMD circuit according to the block diagram depicted in Figure 8.30(a). This FSMD circuit features a datapath comprising three single-precision FP design elements: an FP adder, an FP multiplier, and an FP subtractor arranged in a series configuration. With the assumption that each FP component operates within a single clock cycle, the final output, denoted as "g", will be produced after three clock cycles, following the provision of all four inputs "a-d".

   To coordinate the functioning of the datapath, a FSM serves as the timing controller. The FSM is activated by an enable signal, denoted

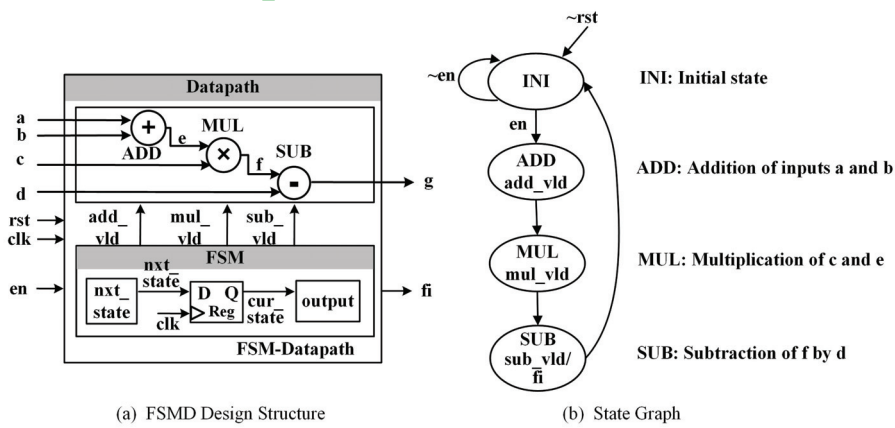

FIGURE 8.30
FP Add-Mul-Sub design with FSMD.

TABLE 8.3
IOs description of FP Add-Mul-Sub design with FSMD.

Name	Direction	Bit Width	Description
clk	Input	1	Clock, 100 MHz
rst	Input	1	Asynchronous reset, 0 valid
en	Input	1	Enable signal
a-d	Input	32	Input data
g	Output	32	Output data
fi	Output	1	Data processing completion indicator

as "en", and produces the completion of data processing utilizing a finish indicator, labeled as "fi". The "en-fi" mechanism follows the "enable-finish" handshaking protocol, facilitating the activation and termination of the FPU engine. A comprehensive list of all inputs and outputs for the circuit is provided in Table 8.3.

According to the design specifications, the state graph of the FSMD is illustrated in Figure 8.30(b). The design engine begins by resetting to the initial state (INI). Upon enabling the circuit, the FSM progresses into the first computational state, dedicated to addition (ADD). Subsequently, it proceeds to the second computational state, focused on multiplication (MUL). Finally, it transitions into the last computational state, which handles subtraction (SUB). During each of these computational states, the validation signals are activated as the outputs of the Moore machine, indicating the validity of the outputs produced by the three individual FP operators. To be specific, the signal "add_vld" confirms the validity of the output "e" from the adder, while the signal "mul_vld" signifies the validity of the output "f" from the multiplier. Furthermore, the signal "sub_vld" denotes the validation of the output "g" from the subtractor.

2) Create a testbench to perform simulations of the FSMD design, and assess its functionality using the provided direct test case illustrated in Figure 8.31. The FSMD is initialized and enabled during the initial clock cycle. Concurrently, the initial data inputs are set as follows: "a" to FP 1.0, "b" to FP 2.0, "c" to FP 3.0, and "d" to FP 0.0. Upon detecting the first asserted finish indicator "fi", the testbench modifies the inputs to the values FP 2.0, 3.0, 2.0, and 1.0 for "a", "b", "c", and "d", respectively. Similarly, subsequent updates to the third and fourth data inputs occur upon the appearance of the

T&F-Not for Distribution

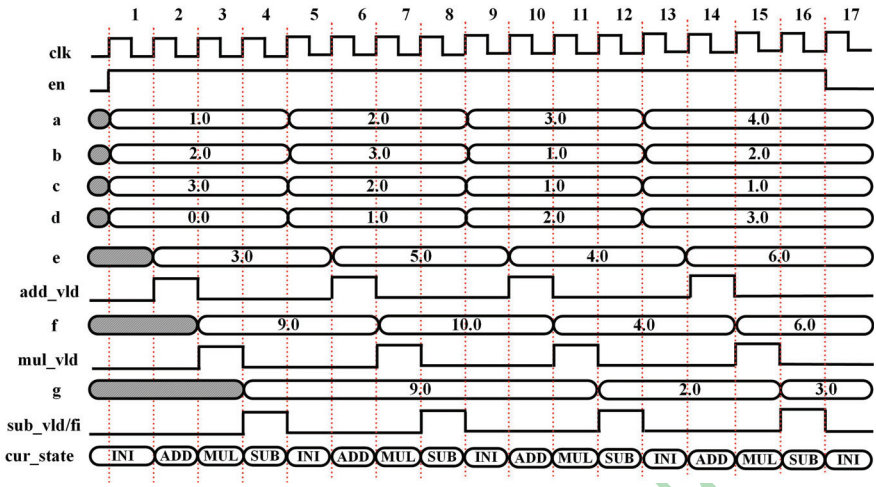

FIGURE 8.31
Timing diagram of FP Add-Mul-Sub with FSMD.

asserted "fi" output. It's important to note that the enable signal intentionally deasserts after receiving the fourth finish indicator, signaling the completion of the final round of operations.

The figure can be further clarified with additional details. The enable signal, represented as "en", signifies the readiness of input data for the FP design engine. This process initiates in the first clock cycle and persists until the completion of the final data package processing. Despite detecting three "fi" signals in clock cycles 4, 8, and 12, indicating the completion of data processing, the "en" signal remains active due to the transmission of four consecutive data packages on the input bus. Furthermore, it's essential to underscore that input data updates only occur upon detecting the finish indicator for each data package processing cycle. Specifically, these updates coincide with clock cycles 5, 9, and 13.

Starting from the first clock cycle, the state machine enters its initial state, INI, and then proceeds sequentially through ADD, MUL, and SUB, covering the entire datapath. This step-by-step process unfolds over clock cycles 1 to 4, resulting in the production of the final output for the first data packet in clock cycle 4. This same procedural pattern is applied to subsequent data packets. For each new data packet, the state machine cycles through its initial states, and the validation signals indicate the progression of operations across clock cycles. Consequently, the validation of the second, third, and fourth data outputs occurs on output "g" during clock cycles 8, 12, and 16, respectively.

T&F - Not for Distribu

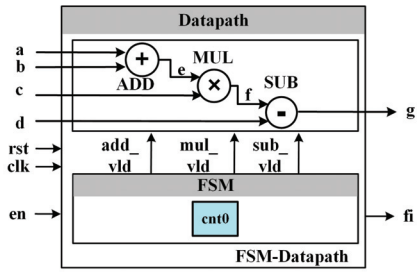

FIGURE 8.32
FP Add-Mul-Sub design with pipeline FP operators.

PBL 19: FP Add-Mul-Sub Design with Pipeline FP Operators

1) As illustrated in Figure 8.32, redesign the circuit in PBL 18 by integrating the pipeline-designed FP operators provided in this book.

   To elaborate, employ pipelined versions of the FP adder, multiplier, and subtractor to process data over the following clock cycles: 13 clock cycles for addition, 10 clock cycles for multiplication, and 13 clock cycles for subtraction, all the way from inputs to outputs. In the timing diagram depicted in Figure 8.33, therefore, the datapath for each set of data inputs spans 37 clock cycles, including one cycle for data input, 13 cycles for FP addition, 10 cycles for FP multiplication, and 13 cycles for FP subtraction. The complete data processing for four sets of data inputs encompasses 148 clock cycles. Rather than utilizing an FSM, you can efficiently control timing by implementing a counter design since the timing control is primarily determined by clock latency.

2) Create a testbench for conducting simulations of the circuit design, and evaluate its functionality using the provided direct test case involving four sets of data inputs in Figure 8.33.

PBL 20: Pipeline FP Add-Mul-Sub Design

1) Redesign the circuit in PBL 19 by implementing a pipeline design structure as depicted in Figure 8.34. Specifically, utilize a pipeline approach to feed four data sets into the design engine, following the timing diagram illustrated in Figure 8.35. The datapath for each

T&F—Not for Distribution

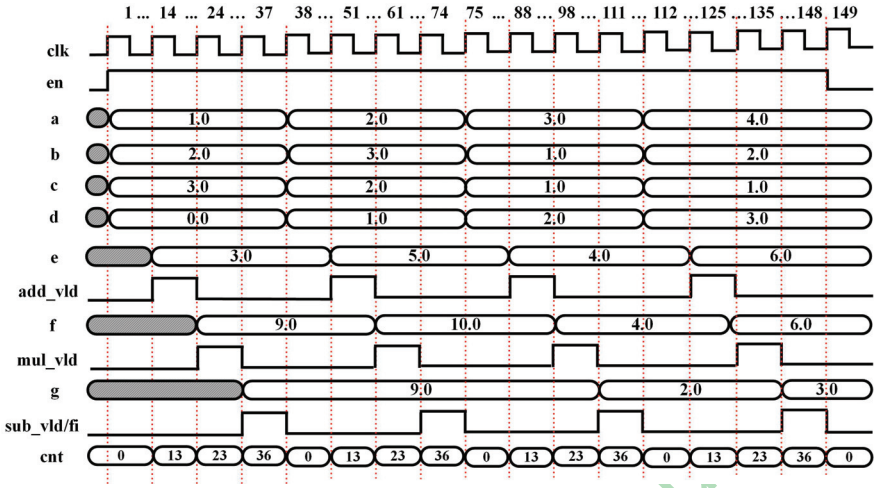

FIGURE 8.33
Timing diagram of FP Add-Mul-Sub design with pipeline FP operators.

set of data inputs spans 37 clock cycles, comprising 1 cycle for data input, 13 cycles for FP addition, 10 cycles for FP multiplication, and 13 cycles for FP subtraction. The complete data processing for these four sets of data inputs extends over 40 clock cycles, with an additional three clock cycles in a pipelined output manner. To design the timing controller effectively, it will be necessary to employ a minimum of four counters, with each counter responsible for tracking one data set.

2) Create a testbench for conducting simulations of the circuit design, and evaluate its functionality using the provided direct test case involving four sets of data inputs in Figure 8.35.

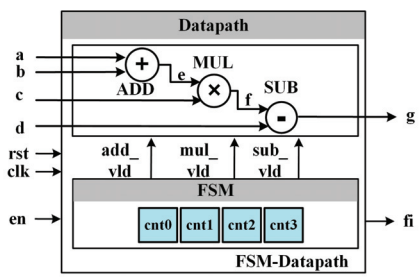

FIGURE 8.34
Pipeline FP Add-Mul-Sub design.

T &F - Not for Distribu

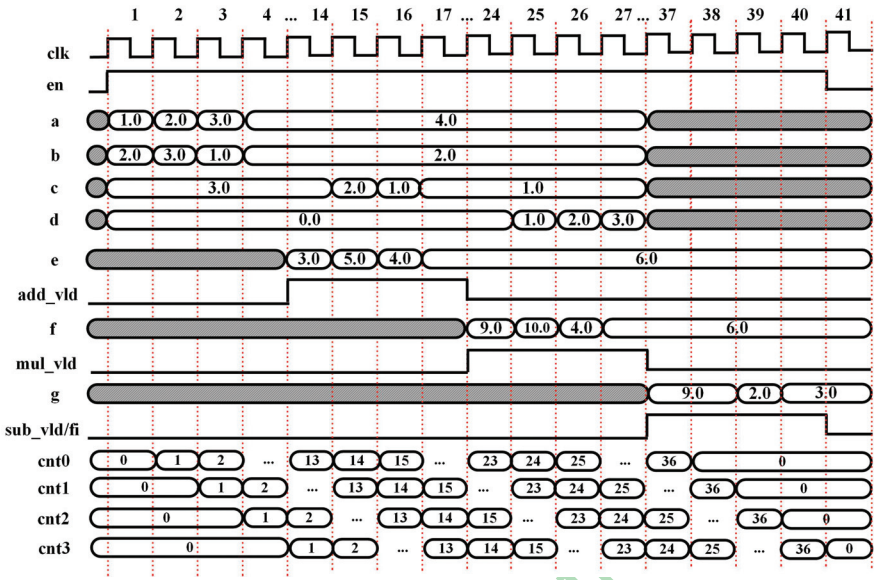

FIGURE 8.35
Timing diagram of pipeline FP Add-Mul-Sub design.

T&F - Not for Distr

PBL 21: Parallel and Pipeline FP Add-Mul-Sub Design

1) Redesign the circuit in PBL 20 by implementing a parallel design structure. In more detail, employ the dual-core design approach to feed the four data sets into the design engine, as exemplified in the design structure shown in Figure 8.36(a). When considering the timing diagram, assess the number of clock cycles required for data processing in this parallel setup.

2) Create a testbench for conducting simulations of the dual-core design, and evaluate its functionality using the provided direct test case involving four sets of data inputs in Figure 8.35. It's worth noting that the dual-core design requires two clock cycles to input the four sets of data, in contrast to the four clock cycles needed in PBL 20.

3) Utilize the quad-core design approach to feed the four data sets into the design engine, as exemplified in the design structure shown in Figure 8.36(b). When considering the timing diagram, assess the number of clock cycles required for data processing in the quad-core setup.

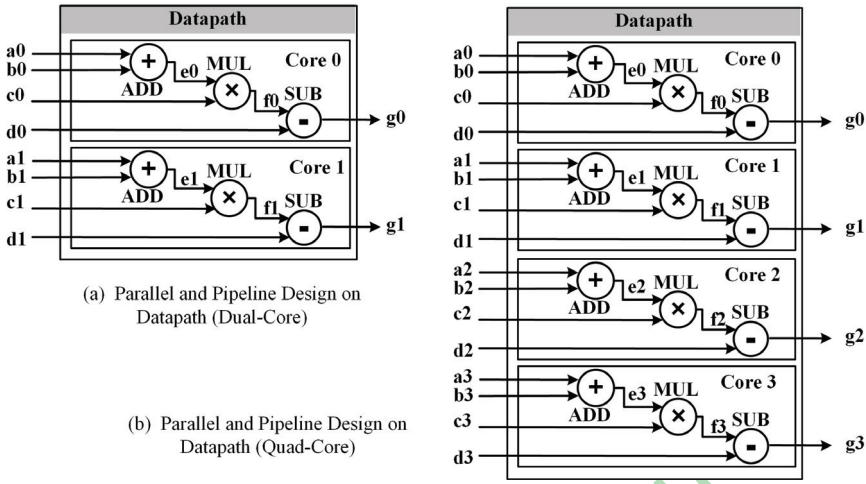

FIGURE 8.36
Parallel FP datapath.

4) Create a testbench for conducting simulations of the quad-core design and evaluate its functionality using the provided direct test case, which involves four sets of data inputs as shown in Figure 8.35. It's noteworthy that the quad-core design requires only one clock cycle to input the four sets of data, in contrast to the four clock cycles needed in PBL 20.

T &F--Not for Distribu